Fundação Escola de Comércio Álvares Penteado

Ciência da Computação 4º Semestre

Projeto Interdisciplinar: IoT e Robótica

Eduardo Chen Zou – 24026559
Lucas Kenichi Soares – 24026179
Pedro Della Rosa Antônio – 24026018
Pedro Dimitry Zyrianoff – 24026165

São Paulo – 2026

Projeto: Smart Tênis — Alvo a Laser para Treino Autônomo

1. Introdução e objetivo da entrega

1.1 Recapitulação da Entrega 1

A Entrega 1 definiu a arquitetura do Smart Tênis — mira a laser em dois eixos com classificação de acerto/erro por visão computacional — e validou o mecanismo físico de posicionamento do laser (servos pan/tilt controlados pelo Arduino Uno), além da comunicação Wi-Fi/HTTP entre o aplicativo Android e o servidor Flask, já operacional naquela fase. Ficaram definidos como fora de escopo daquela etapa a implementação completa do algoritmo de visão computacional, a conclusão do aplicativo mobile e os testes de campo com calibração fina.

1.2 Objetivo desta entrega

Esta Entrega 2 apresenta a implementação final do sistema, evidenciando a integração entre os cinco blocos definidos na Entrega 1 — sensores, processamento, atuadores, comunicação e registro — e valida técnica e funcionalmente a solução planejada. Em particular, esta etapa cobre o que estava fora do escopo anterior: o módulo de visão computacional completo (incluindo um pipeline de treino de um detector de bola por rede neural, YOLOv8), a integração ponta a ponta entre câmeras, Arduino, servidor Flask e aplicativo Android, e os níveis de dificuldade do laser automático (fácil, médio, difícil).

2. Arquitetura final implementada

A arquitetura final mantém os cinco blocos propostos na Entrega 1, agora todos implementados e integrados de ponta a ponta. A principal evolução está no bloco de Processamento: o módulo de visão computacional, que na Entrega 1 era apenas especificado, foi implementado em duas versões — uma detecção por cor (HSV), mais simples, e um detector por rede neural (YOLOv8) treinado com frames reais capturados das próprias câmeras da montagem física, que é o usado na versão atual do projeto.

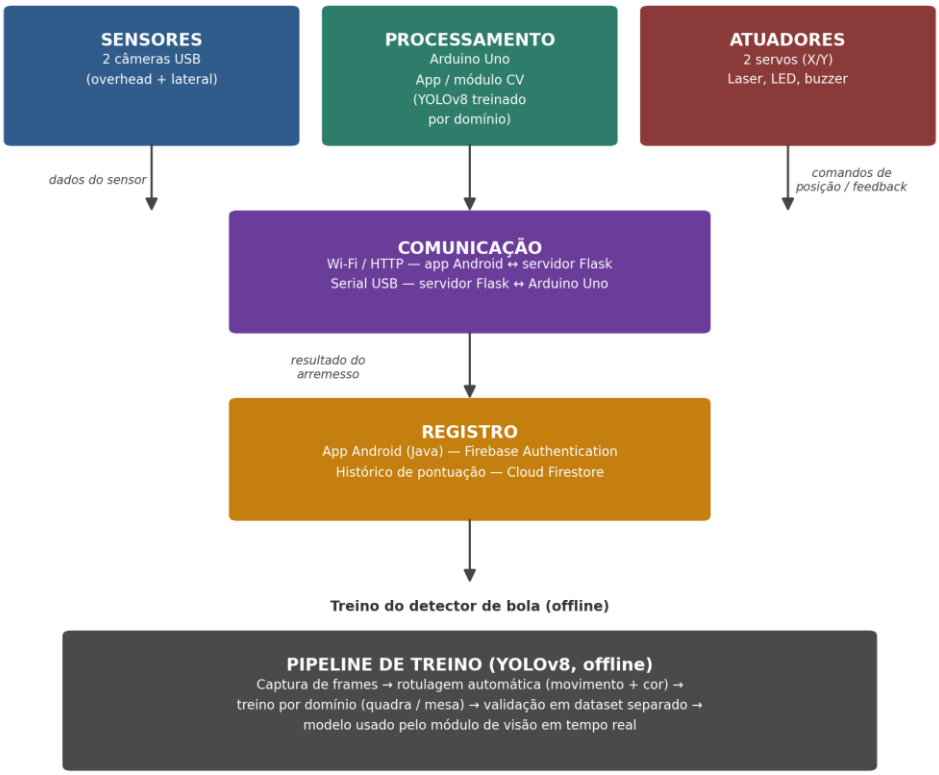


Figura 1 — Arquitetura final do sistema Smart Tênis, com o pipeline de treino do detector de bola.

Bloco	Composição (implementado)
Sensores	2 câmeras USB (uma overhead, uma lateral), cada uma identificada por ID estável e aberta via backend MSMF no Windows
Processamento	Arduino Uno (controle em tempo real dos atuadores) + módulo de visão computacional (detector por cor OU YOLOv8 treinado por domínio)
Atuadores	2 servos (eixos X e Y), módulo laser, 2 LEDs indicadores, buzzer de alerta
Comunicação	Wi-Fi/HTTP entre o app Android e o servidor Flask (Python); Serial USB entre o Flask e o Arduino, com reconexão automática
Registro	App Android nativo (Java) com login/cadastro via Firebase Authentication e histórico de pontuação no Cloud Firestore

3. Validação técnica e funcional da integração

Cada bloco foi testado isoladamente e depois em conjunto, seguindo o fluxo de dados definido na Entrega 1 (entrada → processamento → saída → registro):

- Arduino: o firmware aceita tanto comandos incrementais (A/D/W/S, passos de 5°) quanto posicionamento absoluto (G<x>,<y>, silencioso, usado pelo laser automático a 20 Hz) e responde a consultas de posição (P), permitindo que o servidor detecte automaticamente qual firmware está gravado.

- Servidor Flask: expõe os endpoints /status, /comando, /jogo/iniciar, /jogo/parar, /jogo/modo-manual, /laser/status, /acerto, /visao/heartbeat e /acertos, e reconecta sozinho a porta serial caso o cabo USB solte durante o jogo.
- Laser automático por dificuldade: nível fácil mantém o laser centralizado; médio descreve um círculo lento (período de 10 s); difícil sorteia um novo alvo a cada 2 s dentro de uma faixa angular, com transição suave — cada partida expira sozinha após 70 s por segurança.
- Visão computacional: as duas câmeras foram testadas em conjunto, com triangulação da posição da bola e envio do resultado via POST /acerto; o módulo YOLOv8 foi testado em tempo real com o script de detecção de impacto, usando as mesmas câmeras da montagem física.
- App Android: login/cadastro, tela de jogo com os 3 níveis de dificuldade, histórico de pontuação e controle manual do laser foram testados conectados ao servidor Flask rodando na mesma rede Wi-Fi.

4. Visão computacional: pipeline de treino e resultados

Por limitação de espaço físico para os testes, o projeto foi adaptado para uma miniatura de mesa (mesma arquitetura, escala reduzida), o que exigiu capturar um dataset próprio: frames das duas câmeras durante sessões de jogo, rotulados automaticamente por um algoritmo que combina detecção de movimento (subtração de fundo) com a cor característica da bola de tênis, para distinguir a bola de outros objetos em movimento na cena (mão do jogador, dobras do pano de fundo).

O treino usa YOLOv8 (Ultralytics), continuando sempre do melhor checkpoint já treinado para aquele domínio (quadra real ou miniatura de mesa), com uma pasta de validação separada do treino para evitar métricas infladas. Cada câmera contribui com seu próprio conjunto de frames e rótulos.

Rodada de treino	Precisão	Recall	mAP50
1ª rodada (domínio quadra)	—	—	0,435
2ª rodada	—	—	0,324
3ª rodada	—	—	0,381
4ª rodada (em andamento — época 4/30)	0,17–0,21	0,22–0,45	0,14–0,18

O treino da 4ª rodada, com um dataset bem maior (capturado especificamente para melhorar a câmera overhead, que apresentava escassez de rótulos positivos nas rodadas anteriores), ainda

está em andamento no momento da elaboração deste documento; os valores de mAP50 acima são das primeiras épocas e tendem a melhorar à medida que o treino avança — nas rodadas anteriores, o melhor resultado apareceu por volta da época 10 a 13.

5. Requisitos funcionais e não funcionais — status

ID	Descrição	Status
RF01	Projetar um ponto de laser visível como alvo de treino	Atendido
RF02	Ajustar a posição do laser em dois eixos independentes (X e Y)	Atendido
RF03	Detectar, via visão computacional, a posição da bola após o arremesso	Atendido
RF04	Classificar automaticamente cada arremesso como acerto ou erro	Atendido
RF05	Indicar o resultado do arremesso por LED e por som (buzzer)	Atendido
RF06	Registrar a pontuação de cada arremesso no aplicativo mobile	Atendido
RF07	Permitir login e cadastro do jogador no aplicativo	Atendido
RF08	Exibir o histórico de pontuação do jogador	Atendido
RF09	Oferecer níveis de dificuldade com movimento automático do laser	Atendido

ID	Descrição	Status
RNF01	Feedback (LED/buzzer) em menos de 1 s após a detecção	Atendido
RNF02	Margem de erro da classificação compatível com o tamanho do alvo	Em ajuste — depende do treino do detector
RNF03	Laser de baixa potência (classe 1 ou 2)	Atendido
RNF04	Ajuste de posição simples, sem conhecimento técnico	Atendido
RNF05	Conjunto físico compacto e portátil	Atendido (miniatura de mesa)
RNF06	Calibração dos eixos X/Y mantida durante toda a sessão	Atendido
RNF07	Latência app ↔ Arduino abaixo de 200 ms na mediana	Atendido

6. Dificuldades técnicas e decisões de projeto

- Instabilidade de índice de câmera no Windows: o backend padrão (DirectShow) reatribuía os IDs das câmeras USB de forma inconsistente entre execuções; a solução foi priorizar o backend MSMF, que se mostrou estável nos testes repetidos.
- Escassez de rótulos positivos na câmera overhead: o rotulador automático por movimento (MOG2) absorvia a bola parada ou lenta no modelo de fundo, gerando

poucos exemplos positivos para aquela câmera; a solução foi capturar um dataset maior e mais variado, priorizando sessões com arremessos lentos próximos ao gol.

- Adaptação de escala: por limitação de espaço, o projeto migrou de uma simulação em escala de quadra real para uma miniatura de mesa, o que exigiu recapturar o dataset de treino e ajustar as tolerâncias de distância no detector de impacto.
- Reconexão do Arduino: quedas momentâneas do cabo USB derrubavam a conexão serial; o servidor Flask passou a detectar a falha e reabrir a porta automaticamente, sem precisar reiniciar o servidor.

7. Evidências de funcionamento (foto e vídeo)

[Inserir aqui fotos e/ou vídeo demonstrando o sistema completo em funcionamento: as duas câmeras conectadas, o laser se movendo automaticamente em pelo menos um nível de dificuldade, e o aplicativo Android exibindo o resultado de um arremesso em tempo real.]

8. Próximos passos

- Concluir o treino do detector YOLOv8 da câmera overhead e validar ao vivo com as duas câmeras.
- Ajustar a tolerância de classificação de acerto/erro (RNF02) conforme a precisão real do modelo treinado.
- Testes de campo adicionais com jogadores reais.

9. Conclusão

Esta entrega integra, pela primeira vez, todos os blocos do Smart Tênis em um sistema funcional de ponta a ponta: o laser se move automaticamente conforme o nível de dificuldade escolhido no aplicativo, o Arduino comanda os atuadores em tempo real, o servidor Flask faz a ponte entre o app e o hardware, e o módulo de visão computacional — agora incluindo um detector treinado por rede neural — classifica os arremessos e alimenta o histórico de pontuação do jogador. O item em aberto é a qualidade do detector da câmera overhead, cujo treino está em andamento; o backlog prioriza concluir essa etapa antes dos testes de campo finais.